

Politechnika Warszawska

W Y D Z I A Ł E L E K T R Y C Z N Y



Dokumentacja Projektowa

Daniel Khadarouski

Indeks:347888

10 Kwietnia 2026 r.

Spis treści

1	Cel dokumentacji	3
2	Wejście i wyjście programu	3
2.1	Format danych wejściowych	3
2.2	Format danych wyjściowych	4
3	Argumenty programu (CLI)	4
3.1	Składnia polecenia	4
3.2	Opis parametrów	4
4	Algorytmy	4
4.1	Spectral Layout (Układ spektralny)	5
4.2	Algorytm Fruchtermana-Reingolda	5
4.3	Porównanie algorytmów	5
5	Obsługa błędów i sytuacje wyjątkowe	5
5.1	Katalog komunikatów błędów	5
5.2	Kody powrotu (Exit Codes)	6
6	Podział na moduły	6
6.1	Struktura programu	6
6.2	Struktura modułowa projektu	6
6.2.1	Moduł modelu danych (<code>graph.h</code> , <code>graph.c</code>)	6
6.2.2	Moduł parsera danych (<code>parser.h</code> , <code>parser.c</code>)	7
6.2.3	Moduł algorytmów układu (<code>layout.h</code> , <code>layout.c</code>)	7
6.2.4	Moduł zapisu wyników (<code>write.h</code> , <code>write.c</code>)	7
6.2.5	Główny moduł sterujący (<code>main.c</code>)	7
7	Podsumowanie i wnioski końcowe	7
7.1	Kluczowe aspekty współpracy	7
7.2	Dalszy rozwój i potencjał modułu	8

1 Cel dokumentacji

Niniejsza dokumentacja powstała w celu ułatwienia współpracy między zespołami oraz uproszczenia procesu integracji modułu napisanego w języku C do aplikacji Java. Zamiast analizy całego kodu źródłowego, zebrano tutaj kluczowe informacje, które pozwolą Państwu na szybkie wdrożenie projektu.

Głównymi założeniami opracowania są:

- **Szybki start:** Przedstawienie gotowych poleceń i flag wymaganych do uruchomienia programu.
- **Zrozumienie mechaniki:** Wyjaśnienie działania algorytmów spektralnych, co pomoże w zaprojektowaniu logiki wizualizacji w bibliotece Swing.
- **Bezproblemowa wymiana danych:** Precyzyjny opis formatów wejścia i wyjścia, zapewniający kompatybilność między modułami.
- **Przewidywalność:** Opis sytuacji wyjątkowych i komunikatów błędów ułatwiający diagnostykę problemów.

Mam nadzieję, że dokument ten zaoszczędzi Państwu czasu podczas implementacji warstwy graficznej.

2 Wejście i wyjście programu

2.1 Format danych wejściowych

Program przyjmuje pliki tekstowe reprezentujące strukturę grafu w formie listy krawędzi. Pierwsza linia pliku musi zawierać dwie liczby całkowite: liczbę wierzchołków oraz liczbę krawędzi. Każda kolejna linia opisuje pojedynczą krawędź w formacie:

<etykieta> <id_źródła> <id_celu> <waga/długość>

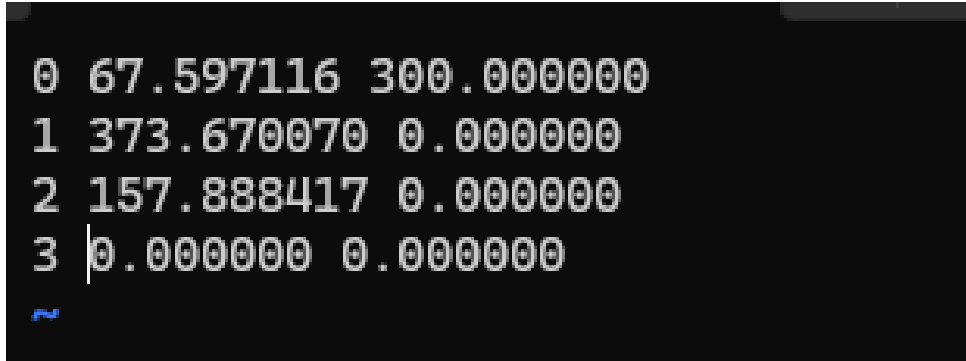
```
4 4
AB 1 2 1
BC 2 3 1
CD 3 4 1
DB 4 2 1.407
```

Rysunek 1: Przykład struktury pliku wejściowego (graf o 4 wierzchołkach i 4 krawędziach)

2.2 Format danych wyjściowych

Wynikiem działania programu są współrzędne dwuwymiarowe (x, y) obliczone dla każdego wierzchołka. Każda linia pliku wyjściowego odpowiada jednemu wierzchołkowi:

<id_wierzchołka> <współrzędna_x> <współrzędna_y>



```
0 67.597116 300.000000
1 373.670070 0.000000
2 157.888417 0.000000
3 0.000000 0.000000
```

Rysunek 2: Wygenerowane współrzędne wierzchołków gotowe do wizualizacji

3 Argumenty programu (CLI)

Program obsługiwany jest z poziomu wiersza poleceń. Użytkownik konfiguruje parametry za pomocą flag.

3.1 Składnia polecenia

Kompilacja programu (przykładowa):

```
gcc main.c graph.c layout.c parser.c write.c -o test -lm
```

Uruchomienie programu:

```
./test -i input.txt -o output.txt -a 1
```

3.2 Opis parametrów

- i **Plik wejściowy** — ścieżka do pliku zawierającego strukturę grafu.
- o **Plik wyjściowy** — ścieżka do pliku, w którym zostaną zapisane współrzędne.
- a **Algorytm** — wybór metody układania grafu (np. 1 dla Fruchterman-Reingold, 2 dla Spectral Layout).

4 Algorytmy

W programie zaimplementowano dwa podejścia do wizualizacji:

4.1 Spectral Layout (Układ spektralny)

Opis działania: Algorytm wykorzystuje wektory własne macierzy Laplaciana grafu $L = D - A$. Współrzędne są wyznaczane na podstawie drugiego i trzeciego najmniejszego wektora własnego.

- **Zastosowanie:** Wykrywanie klastrów i symetrii w dużych zbiorach danych.
- **Zalety:** Metoda deterministyczna, bardzo szybka i stabilna matematycznie.

4.2 Algorytm Fruchtermana-Reingolda

Opis działania: Algorytm siłowy (force-directed), w którym krawędzie działają jak sprężyny przyciągające, a wierzchołki jak ładunki odpychające się. Proces dąży do minimalizacji energii układu.

- **Zastosowanie:** Tworzenie estetycznych i czytelnych map grafów.
- **Zalety:** Równomierne rozłożenie wierzchołków.

4.3 Porównanie algorytmów

Cecha	Spectral Layout	Fruchterman-Reingold
Metoda	Algebraiczna	Fizyczna (symulacja sił)
Czas pracy	Zależny od obliczeń wektorów własnych	Zależny od liczby iteracji
Determinizm	Tak	Nie (zależny od pozycji startowej)
Estetyka	Struktura matematyczna	Intuicyjny, organiczny wygląd

Tabela 1: Porównanie charakterystyki algorytmów

5 Obsługa błędów i sytuacje wyjątkowe

Program został zaprojektowany z myślą o stabilności, dlatego proces walidacji obejmuje zarówno parametry wejściowe, jak i strukturę samego grafu. W przypadku wystąpienia błędu, program przerywa pracę i wysyła czytelny komunikat do strumienia błędów standardowych (`stderr`).

5.1 Katalog komunikatów błędów

Błędne argumenty wywołania — Występuje, gdy użytkownik nie poda ścieżki do pliku wejściowego (flaga `-i`) lub wyjściowego (flaga `-o`). Bez określenia obu tych lokalizacji program nie może zainicjować procesu przetwarzania.

Problemy z dostępem do plików — Pojawia się, gdy podana ścieżka jest nieprawidłowa, plik wejściowy nie istnieje lub system operacyjny blokuje dostęp do odczytu danych. Analogiczny błąd występuje przy braku uprawnień do zapisu w folderze docelowym dla pliku wyjściowego.

Niewłaściwy format danych — Błąd ten jest zgłaszany, gdy zawartość pliku tekstowego jest niezgodna ze specyfikacją techniczną. Przykładem może być obecność liter w miejscu, gdzie oczekiwana jest liczba wierzchołków lub waga krawędzi.

Brak spójności grafu — Sytuacja wyjątkowa wykrywana przed uruchomieniem obliczeń spektralnych. Ponieważ algorytm wymaga grafu połączonego, wykrycie osobnych komponentów uniemożliwia poprawne wyznaczenie układu współrzędnych.

Nieznany algorytm przetwarzania — Występuje w momencie podania parametru wyboru algorytmu spoza zdefiniowanego zakresu. Obecnie obsługiwane są dwie metody: **1** dla algorytmu Fruchtermana-Reingolda oraz **2** dla układu spektralnego.

5.2 Kody powrotu (Exit Codes)

W celu ułatwienia automatyzacji (np. przy wywoływaniu modułu z poziomu Javy), program zwraca następujące kody:

- 0 — Sukces: Obliczenia zakończone pomyślnie.
- 1 — Błąd krytyczny: Szczegóły błędu wypisane w konsoli.

6 Podział na moduły

6.1 Struktura programu

Tutaj przedstawiony jest spis modułów całego programu:

danikchodo Update input.txt		b8d6cd3 · now	67 Commits
README.md	update README.md	last month	
funkcj.pdf	Add files via upload	last month	
graph.c	graph.c	last month	
graph.h	Update graph.h	1 hour ago	
input.txt	Update input.txt	now	
layout.c	Update layout.c	36 minutes ago	
layout.h	Update layout.h	last week	
main.c	Update main.c	32 minutes ago	
parser.c	Update parser.c	2 hours ago	
parser.h	parser.h	last month	
spectral_layout_implementacja-3.pdf	Add files via upload	3 weeks ago	
write.c	Update write.c	last week	
write.h	write.h	last month	

6.2 Struktura modułowa projektu

6.2.1 Moduł modelu danych (graph.h, graph.c)

Jest to fundament projektu, definiujący sposób reprezentacji grafu w pamięci operacyjnej.

- **Struktury danych:** Definiuje typy `node`, `edge` oraz `graph`.
- **Zarządzanie pamięcią:** Zawiera funkcje `init_graph` oraz `free_graph`.

6.2.2 Moduł parsera danych (`parser.h`, `parser.c`)

Ten moduł pełni rolę interfejsu wejściowego programu.

- **Wczytywanie danych:** Funkcja `readgraph` przetwarza plik tekstowy.
- **Abstrakcja:** Wydzielenie tego modułu pozwala na uniezależnienie logiki od formatu pliku.

6.2.3 Moduł algorytmów układu (`layout.h`, `layout.c`)

Serce obliczeniowe programu.

- **Fruchterman-Reingold:** Algorytm siłowy.
- **Spectral Layout:** Metoda matematyczna wykorzystująca macierz Laplasa.

6.2.4 Moduł zapisu wyników (`write.h`, `write.c`)

Odpowiada za eksport danych.

- **Formaty wyjściowe:** Obsługuje zapis tekstowy (`savetxt`) oraz binarny (`savebin`).

6.2.5 Główny moduł sterujący (`main.c`)

Pełni rolę koordynatora całego procesu.

- **Interfejs CLI:** Obsługuje argumenty wiersza poleceń.
- **Kontrola cyklu życia:** Sekwencyjnie wywołuje parser, algorytm i zapis.

7 Podsumowanie i wnioski końcowe

Niniejsze opracowanie stanowi kompletny przewodnik po module układania grafów, przygotowany z myślą o płynnej współpracy międzyzespołowej. Zgodnie z przyjętymi założeniami, dokumentacja została stworzona w taki sposób, aby umożliwić programistom warstwy wizualnej (Java) pełne wykorzystanie potencjału silnika obliczeniowego napisanego w języku C, bez konieczności zgłębiania niskopoziomowych szczegółów implementacji.

7.1 Kluczowe aspekty współpracy

Podczas tworzenia projektu szczególną wagę przywiązano do trzech filarów profesjonalnej dokumentacji technicznej:

- **Perspektywa użytkownika:** Treść została sformułowana pod kątem potrzeb innego zespołu. Skupiono się na interfejsie CLI oraz strukturach danych, co minimalizuje barierę wejścia i pozwala na szybkie wdrożenie modułu do głównego ekosystemu aplikacji.
- **Praktyczna demonstracja:** Zamiast teoretycznych opisów, w dokumencie zawarto konkretne przykłady plików wejściowych oraz wyjściowych. Pozwala to na błyskawiczne przeprowadzenie testów jednostkowych i weryfikację poprawności generowanych współrzędnych.

- **Techniczna precyzja:** Każda flaga, format liczbowy oraz sytuacja wyjątkowa zostały opisane w sposób jednoznaczny. Precyzyjne zdefiniowanie błędów chroni zespół przed czasochłonnym debugowaniem i nieporozumieniami na etapie integracji.

7.2 Dalszy rozwój i potencjał modułu

Zastosowana architektura modularna (podział na parser, model i layout) nie tylko ułatwia obecną współpracę, ale również otwiera drogę do łatwej rozbudowy programu w przyszłości. Możliwe jest dodanie kolejnych algorytmów (np. Tutte) bez modyfikacji istniejących mechanizmów wejścia/wyjścia.

Podsumowując, dostarczony moduł wraz z niniejszą dokumentacją tworzy stabilną bazę dla aplikacji do wizualizacji grafów. Wierzymy, że wysoki stopień dopracowania formatów wymiany danych oraz jasna struktura komunikatów pozwolą na uniknięcie najczęstszych błędów projektowych i przyczynią się do sukcesu końcowego całego projektu.